

MODULE ALGORITHMIQUE ET STRUCTURES DE DONNÉES

FILES

©B. DERDOURI

CM- FILES Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

1

PLAN

- Introduction
- Exemples & Définition
- Intérêts & applications
- Opérations de bases & Représentation
- Exemple : Evaluation Expression PostFixée
- Implémentation contiguë & opérations
- Implémentation chaînée & opérations

©B. DERDOURI

CM-FILES Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

2

FILES

Introduction

- Les files sont des listes qui correspondent au comportement d'une file d'attente devant un guichet où la gestion est la suivante : le premier arrivée sera le premier servi. (Fisrt In First Out ou simplement FIFO). Les usagers arrivent en queue de file, et sortent de la file à sa tête.
- Ces files d'attente sont d'un usage très répandu dans la programmation système.

EXEMPLE

- une file d'attente à la caisse d'un magasin. Les personnes arrivent une à une et constituent la file et elles partent une à une dans l'ordre d'arrivée après avoir payé.
- Toute insertion ou retrait d'une personne en milieu d'une file est une opération qui pose des problèmes !.



DÉFINITION

- Une file d'attente est une liste linéaire telle que :
- Les insertions (entrées) sont toujours effectuées en queue
- Les suppressions (sorties) sont toujours effectuées en tête



DÉFINITION

- Une file d'attente est une liste linéaire telle que :
- Les insertions (entrées) sont toujours effectuées en queue
- Les suppressions (sorties) sont toujours effectuées en tête



INTÉRÊTS

- Gestion système (imprimante, ...)
- Gestion files d'attentes
- Parcours d'arborescences

APPLICATIONS

- **En système** les files sont utilisées pour gérer tous les processus en attente de ressource système.
- D'une façon générale elles interviennent dans un modèle dès qu'il est question d'une **file d'attente** : systèmes de réservation, gestion de pistes d'aéroport etc.

OPÉRATIONS DE BASE

- **estVideFile** : teste si une file donnée est vide ou pas
- **estPleinFile** : teste si une file donnée est pleine ou pas
- **initFile** : initialise une file
- **sommetFile** : retourne l'élément au sommet de la file sans que la file soit modifiée
- **enfiler** : ajout un nouvel élément au sommet d'une file
- **défiler** : supprime l'élément au sommet d'une file

REPRÉSENTATIONS

- Une file est une liste linéaire, elle peut donc être représentée de deux manières :
 - contiguë
 - chaînée

REMARQUE

- Les différentes primitives (opérations) ne tiennent pas compte de la représentation.
- Seul le corps des primitives, inaccessible à l'utilisateur, dépend de cette représentation.

EXEMPLE

- Transformation d'une expression infixée complètement parenthésée en notation postfixée (polonaise inversée).
- Le résultat de la conversion est rangé dans une file.
- Exemple
- L'expression suivante :
- $(3 * (((5 - 2) / 4) - 1))$ devra être rangé en file 3 5 2 - 4 / 1 - *

REPRÉSENTATION CONTIGUË D'UNE FILE

LANGAGE ALGORITHMIQUE

LANGAGE C

©B. DERDOURI

CM-FILES

Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

13

REPRÉSENTATION CHAÎNÉE D'UNE FILE

```

Typedef ... TElement;
Typedef struct cellule{
    TElement donnee;
    struct cellule *suivant;
} *File;

```

 **typedef Liste File;**

©B. DERDOURI

CM-FILES

Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

14

REPRÉSENTATION CHAÎNÉE D'UNE PILE

File : f		Liste : l
initFile()		initListe(l)
estVideFile(f)		estVideListe(l)
Fonction estPleinFile Pile → Booléenne • Fonction estPleinFile(f) → Booléenne • P.F f: File (E) • Debut • Renvoyer faux • Fin		
©B. DERDOURI CM-FILES Module Algo2 L2-MI_Sem3-U-CERGY 10/23/2022		

15

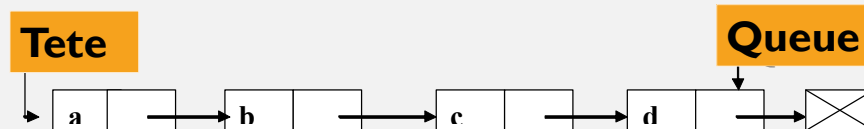
REPRÉSENTATION CHAÎNÉE D'UNE PILE

sommetFile(f)		donneeListe(l)
enfiler(elt, f)		inserQueue(elt, l)
Défiler(f)		suppTete(l)

16

AMÉLIORATION DE LA STRUCTURE

- On utilise un pointeur supplémentaire : le pointeur sur la dernière cellule, ce qui permet d'accélérer l'algorithme d'insertion qui a toujours lieu en fin de liste.



- Gestion d'une file par listes chaînées*

DÉCLARATION LA STRUCTURE EN C

```
typedef struct cellule{
    TElement donnee;
    struct cellule *suivant;
}* Liste ;
```

```
typedef struct {
    Liste tete ;    // pointeur sur la
                  // première cellule
    Liste queue ;  // pointeur sur la
                  // dernière cellule
} File ;
```

OPÉRATIONS DE BASE

Fonction tete File $\rightarrow$ Liste

Fonction teteF(f) $\rightarrow$ Liste

P.F f : File (E)

Debut

Renvoyer f.tete

Fin

Fonction queue File $\rightarrow$ Liste

Fonction queue(f) $\rightarrow$ Liste

P.F f : File (E)

Debut

Renvoyer f.queue

Fin

OPÉRATIONS

• Fonction **initFile** : $\{\} \rightarrow$ File

• Fonction **initFile()** $\rightarrow$ File

• Début

- f.tete $\leftarrow$ initListe()
- f.queue $\leftarrow$ initListe()
- renvoyer f

• FIN

OPÉRATIONS

- **Fonction estVideFile : File $\rightarrow$ Booléenne**
- **Fonction estVideFile(f) $\rightarrow$ Booléenne**
- **P.F f : File (E)**
- **Début**
 - Renvoyer estVideListe(tete(f))
- **FIN**

©B. DERDOURI

CM-FILES

Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

21

OPÉRATIONS

- **Fonction estPleinFile : File $\rightarrow$ Booléenne**
- **Fonction estPleinFile(f) $\rightarrow$ Booléenne**
- **P.F f : File (E)**
- **Début**
 - Renvoyer Faux
- **FIN**

©B. DERDOURI

CM-FILES

Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

22

OPÉRATIONS

- **Fonction sommetFile : File $\rightarrow$ TElement**
- //Pré-condition : la file est non vide
- **Fonction sommetFile(f) $\rightarrow$ Telement**
- P.F f : File (E)
- **Début**
- Renvoyer donneeListe(tete(f))
- **FIN**

OPÉRATIONS

- **Fonction enfiler : TElement x File $\rightarrow$ File**
- //Pré-condition : la file est non plein
- **Fonction enfiler(elt, f) $\rightarrow$ FILE**
- P.F elt : TElement (E) f : File (E)
- **Début**
- Cel $\leftarrow$ allocMemoire(Cellule)
- Cel.donnee $\leftarrow$ elt
- Cel.suivant $\leftarrow$ NULL
- Si (estVideListe(tete))
- f.tete $\leftarrow$ f.queue $\leftarrow$ cel
- else //Insertion en Queue d'une File
- f.queue.suivant $\leftarrow$ cel
- f.queue $\leftarrow$ cel
- fsi
- renvoyer f
- **FIN**

OPÉRATIONS

- **Fonction enfiler : TElement x File → File**
- //Pré-condition : la file est non plein
- **procédure enfiler(elt, f)**
- P.F elt : TElement (E) f : File (E/S)
- **Début**
 - Cel ← allocMemoire(Cellule)
 - Cel.donnee ← elt
 - Cel.suivant ← NULL
 - Si (videListe(tete))
 - f.tete ← f.queue ← cel
 - else //Insertion en Queue d'une File
 - f.queue.suivant ← cel
 - f.queue ← cel
 - fsi
- **FIN**

©B. DERDOURI

CM-FILES

Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

25

OPÉRATIONS

- **Fonction defiler : File → File**
- //Pré-condition : la file est non vide
- **Fonction defiler(f) → File**
- P.F f : File (E/S)
- **Début**
 - tt ← tete(f) // On mémorise la tête
 - Si (tete(f)=queue(f)) alors
 - f.tete ← f.queue ← NULL
 - Sinon
 - f.tete ← suivant(tete(f)) // passage au suivant
 - Fsi
 - libMemoire(tt) // libération de la mémoire
 - // occupée par l'ancienne tête
 - renvoyer(f)
- **FIN**

©B. DERDOURI

CM-FILES

Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

26

OPÉRATIONS

- **Fonction defiler : File $\rightarrow$ File**
- //Pré-condition : la file est non vide
- **procédure défiler(f)**
- P.F f : File (E/S)
- **Début**
 - $tt \leftarrow \text{tete}(f)$ // On mémorise la tête
 - Si $(\text{tete}(f) = \text{queue}(f))$ alors
 - $F.\text{tete} \leftarrow f.\text{queue} \leftarrow \text{NULL}$
 - Sinon
 - $f.\text{tete} \leftarrow \text{suivant}(\text{tete}(f))$ // passage au suivant
 - Fsi
 - libMemoire(tt) // libération de la mémoire
 - // occupée par l'ancienne tête
- **FIN**

©B. DERDOURI

CM-FILES

Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

27

EXERCICES

- Ecrire un algorithme qui transforme une expression infixée complètement parenthésée en notation postfixée (polonaise inversée).
- le résultat de la conversion est rangé dans une file.
- **Exemple**
- L'expression suivante :
- $(3 * (((5 - 2) / 4) - 1))$ devra être rangé en file 3 5 2 – 4 / 1 - *

©B. DERDOURI

CM-FILES

Module Algo2 L2-MI_Sem3-U-CERGY

10/23/2022

28